

AgentSkills完全指南：从入门到精通，建议收藏

文章详细介绍了Anthropic提出的Agent Skills概念，它是Agent->Skills->Tools三层架构中的关键组成部分。Skills是一种包含指令、脚本和资源的有序文件夹，智能体可动态发现并加载，通过渐进式披露原则提高效率。文章解析了Skills的结构、工作原理、开发评估方法及安全注意事项，强调了Skills在弥补大模型使用工具短板、实现特定领域专业能力方面的重要价值。

随着对AI Agent的探索与实践逐步深入，我越来越能感觉到组合：

Agent -> Skills -> Tools (function call 和 MCP)

的有效性，它们各自处在不同的层级，共同可靠地完成任务。

对于Agent和Tools，应用中可以说已经司空见惯，大家已经从理解到实践理解得较透彻了，而对于Skills则不然，它出现比较晚，25年10月份才由Anthropic公开提出来，它的定位也容易让人疑惑，“是不是就是把一部分上下文切出来便于复用？”，诸如此类的问题直到今天在绝大多数人脑中还有待回答。

老祖宗说“纸上得来终觉浅”啊。虽然自媒体炒作这个概念就没停过，但我觉得吧蹭热度的多，内容少了些，我自己构建Jarvis时获得的经验应该还是有些参考价值的，那就写写吧。非常希望获得你的关注，让我们一起成长吧 😊

2025年12月18日，Anthropic已经将Agent Skills 作为跨平台可移植的开放标准发布，详情参考

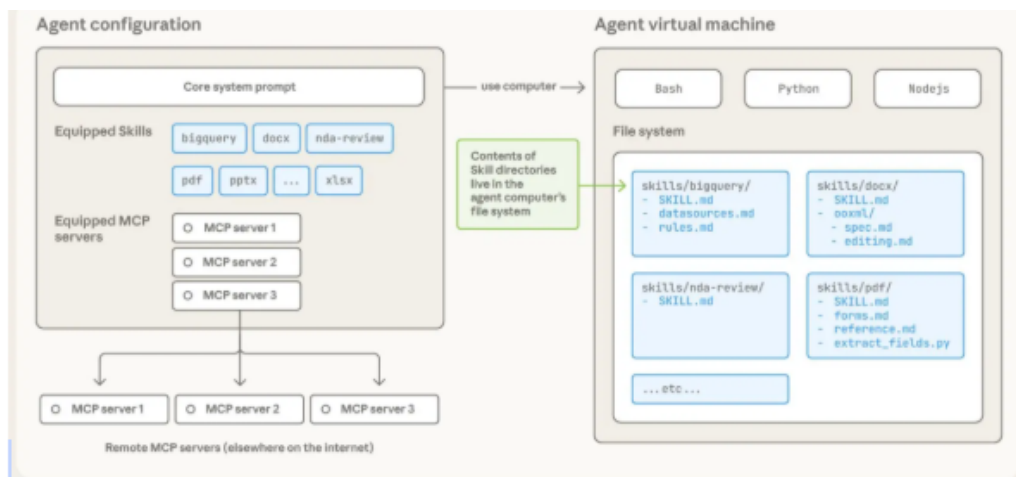
<https://www.agentskills.io>。作为AI应用构建者，你必须学习它了。接下来我会围绕Skills写几篇文章：从它被提出，到成为标准，到Anthropic的最佳实践，最后是我的一些实践真知。这系列的前半部分我会尊重Anthropic的原作，从而勾勒出Skills的成长历程。

下面这篇文章的内容来自Anthropic，它第一次正式介绍了Skills这个概念，并给出使用指引。我在之前的文章中简单介绍过，这次我尽量不遗漏，全面引用其内容，你会看到当初是怎么回事儿。同时我会将我的一点看法放在文章最后，欢迎大家阅读交流。在阅读中请牢记，这是它最初的样子，目前在某些点上可能已经演化了。

随着模型能力的提升，我们现在可以构建能够与完整计算环境交互的通用型智能体。例如，Claude Code 能够通过本地代码执行和文件系统，在多个领域完成复杂任务。但随着这些智能体变得更强大，我们需要更具可组合性、可扩展性和可移植性的方式，为它们赋予特定领域的专业知识。

这促使我们创建了 Agent Skills：即由指令、脚本和资源组成的有序文件夹，智能体可以动态发现并加载这些内容，从而在特定任务上表现得更好。Skills 通过将你的专业知识打包为可组合的资源，扩展了 Claude 的能力，使通用型智能体转变为适合你需求的专用型智能体。

为智能体构建一个 Skill，就像为新员工准备入职指南。现在，任何人都可以通过捕捉和分享流程知识，用可组合的能力让他们的智能体专精于某些领域，而无需为每个用例单独定制智能体。在本文中，我们将解释什么是 Skills，展示其工作原理，并分享构建你自己的 Skill 的最佳实践。



Skill是一个包含 SKILL.md 文件的目录，其中组织了指令、脚本和资源的文件夹，为代理提供额外的能力

Skill 的结构

Skill 是一个包含 SKILL.md 文件的目录，里面有有序的指令、脚本和资源文件，为智能体提供额外能力。

让我们通过一个实际例子来了解 Skills 的应用：Claude 最近上线的文档编辑能力就依赖于某个 Skill。Claude 已经很擅长理解 PDF，但直接操作 PDF（如填写表单）的能力有限。这个 PDF Skill 让 Claude 获得了这些新能力。

最简单的 Skill 是一个包含 SKILL.md 文件的目录。该文件必须以 YAML 前言开头，包含一些必要的元数据：名称和描述。启动时，智能体会将每个已安装 Skill 的名称和描述预加载到系统提示中。

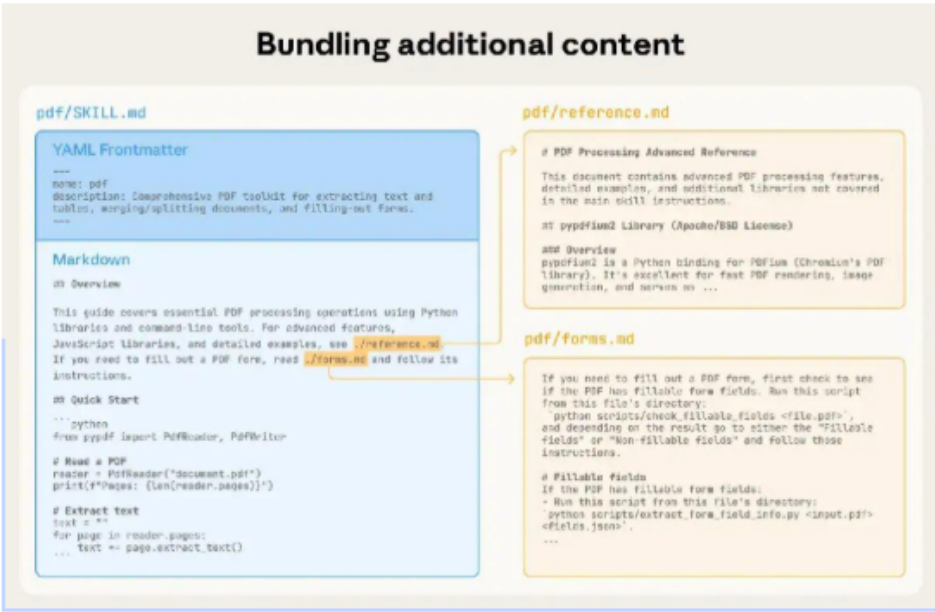
这些元数据是渐进式披露的第一层：它为 Claude 提供了足够的信息，以便在不加载全部内容的情况下判断何时使用某个 Skill。文件正文则是第二层细节。如果 Claude 认为该 Skill 与当前任务相关，它会通过读取完整的 SKILL.md 来加载该 Skill。



SKILL.md 文件必须以 YAML 前置数据开头，其中包含文件名和描述，这些内容会在启动时加载到其系统提示中。

随着 Skill 变得更复杂，可能包含太多内容，无法全部放入一个 SKILL.md 文件，或者有些内容只在特定场景下相关。这时，Skill 可以在目录中捆绑其他文件，并在 SKILL.md 中引用它们。这些附加文件就是第三层（及更深层次）的细节，Claude 只会在需要时选择性地读取。

在下方的 PDF Skill 示例中，SKILL.md 引用了另外两个文件（reference.md 和 forms.md），这些文件与核心 SKILL.md 一起打包。通过将表单填写指令移到 forms.md 文件，Skill 作者可以保持核心内容简洁，并相信 Claude 只会在填写表单时读取 forms.md。



你可以通过添加更多文件为你的技能引入更多上下文，这些文件随后可以根据系统提示由 Claude 触发。

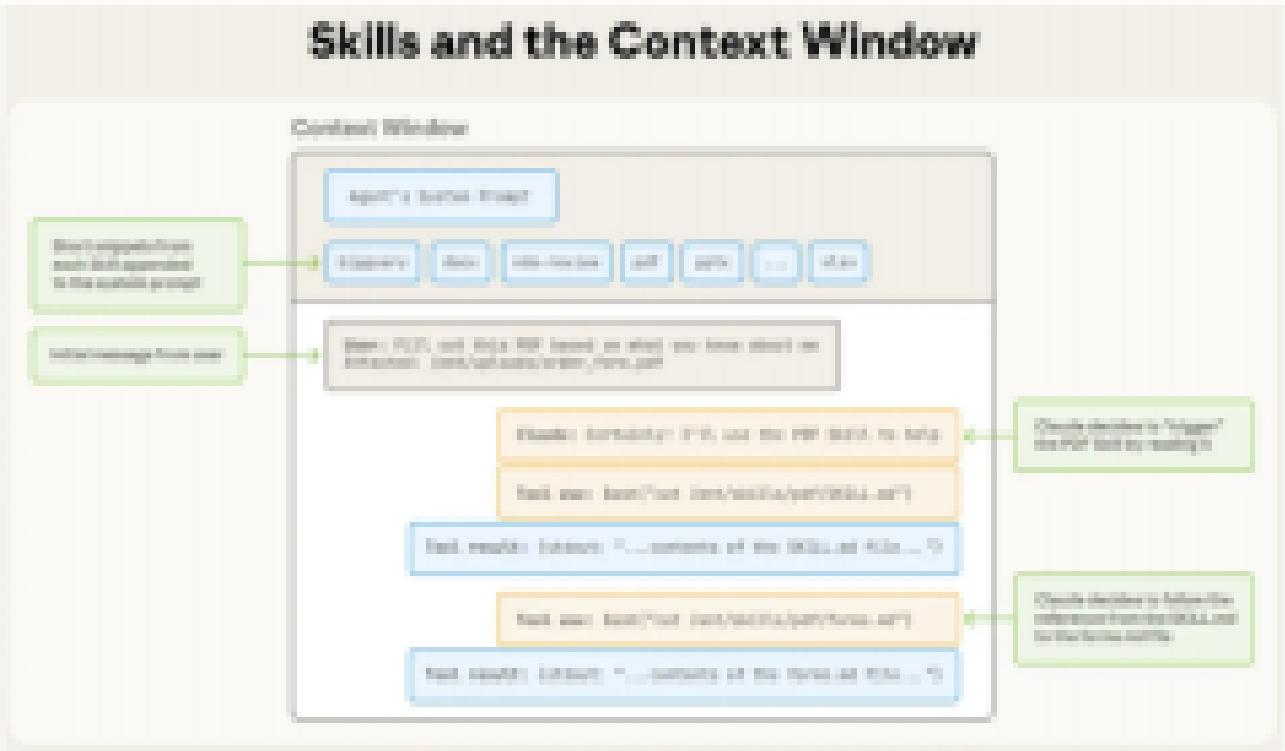
渐进式披露是让 Agent Skills 灵活且可扩展的核心设计原则。就像一本有目录、章节和详细附录的手册，Skills 让 Claude 只在需要时加载信息：

Level	File	Context Window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (Markdown)	Loaded when Skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as-needed by Claude	unlimited*

拥有文件系统和代码执行工具的智能体，在处理特定任务时无需将整个 Skill 全部读入上下文窗口。这意味着可以打包进 Skill 的上下文量实际上是无限的。

Skills 与上下文窗口

下图展示了当用户消息触发 Skill 时，上下文窗口的变化过程。



原文为流程图，简述如下

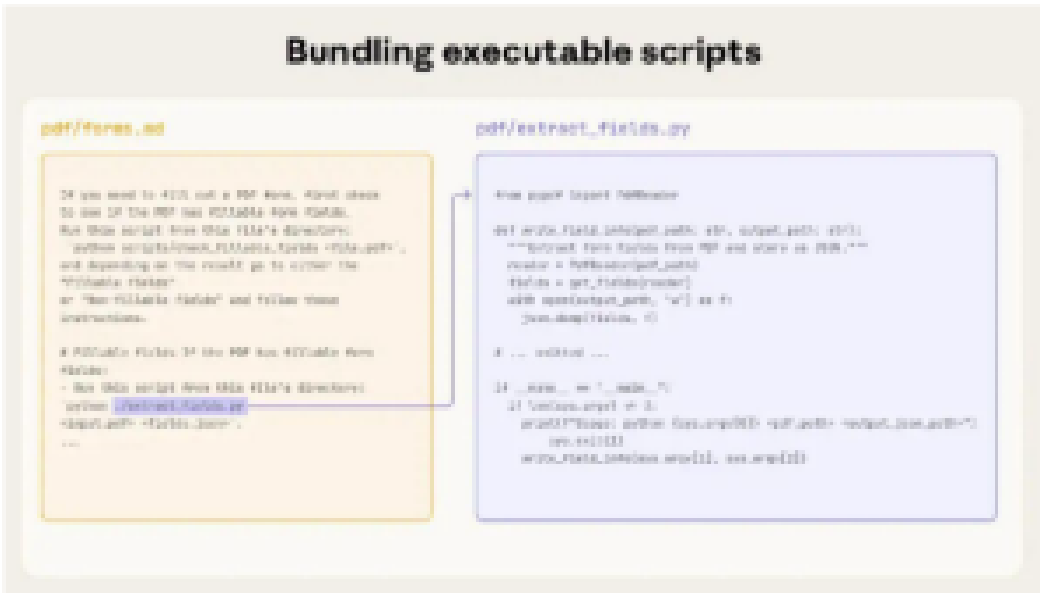
- 起始时，上下文窗口包含核心系统提示、所有已安装 Skill 的元数据，以及用户的初始消息；
- Claude 通过调用 Bash 工具读取 pdf/SKILL.md 的内容，触发 PDF Skill；
- Claude 选择读取 Skill 捆绑的 forms.md 文件；
- 最后，Claude 在加载了 PDF Skill 的相关指令后，继续完成用户任务。

Skills 与代码执行

Skills 还可以包含供 Claude 按需执行的代码。

大型语言模型在许多任务上表现出色，但某些操作更适合用传统代码执行。例如，通过生成 token 排序列表比直接运行排序算法要昂贵得多。除了效率问题，许多应用还需要只有代码才能提供的确定性可靠性。

在我们的示例中，PDF Skill 包含一个预写的 Python 脚本，用于读取 PDF 并提取所有表单字段。Claude 可以在不将脚本或 PDF 加载到上下文中的情况下运行该脚本。由于代码具有确定性，这一流程既一致又可复现。



Skills 还可以根据任务性质，包含供 Claude 按需执行的工具型代码。

开发与评估 Skills

以下是编写和测试 Skills 的一些有用建议：

- 从评估开始：通过让智能体在代表性任务上运行，观察其在哪些地方表现不佳或需要更多上下文，然后逐步构建 Skills 以弥补这些不足。
- 结构化以便扩展：当 SKILL.md 文件变得难以管理时，将内容拆分到不同文件并引用。如果某些上下文互斥或很少同时使用，分开路径可以减少 token 使用。代码既可作为可执行工具，也可作为文档。应明确 Claude 是直接运行脚本还是作为参考读入上下文。
- 从 Claude 的视角思考：监控 Claude 在实际场景中如何使用你的 Skill，并根据观察结果迭代：注意意外的行为轨迹或对某些上下文的过度依赖。特别关注 Skill 的名称和描述，Claude 会用它们来决定是否在当前任务中触发 Skill。
- 与 Claude 一起迭代：在与 Claude 协作完成任务时，让 Claude 把成功的方法和常见错误整理进 Skill 的上下文和代码。如果 Claude 在使用 Skill 完成任务时偏离目标，让它自我反思哪里出了问题。这个过程有助于你发现 Claude 实际需要哪些上下文，而不是事先猜测。

使用 Skills 时的安全注意事项

Skills 通过指令和代码为 Claude 提供新能力。这使它们很强大，但也意味着恶意 Skill 可能在使用环境中引入漏洞，或让 Claude 泄露数据、执行非预期操作。

我们建议只从可信来源安装 Skills。如果从不太可信的来源安装 Skill，务必在使用前彻底审查。首先阅读 Skill 捆绑的所有文件内容，了解其作用，特别关注代码依赖和捆绑的资源（如图片或脚本）。同样，注意 Skill 中可能让 Claude 连接到不可信外部网络的指令或代码。